



CSC220 · WEB DEVELOPMENT II · WEEK 1 · SESSION 2

DOM Manipulation & Events

Making pages interactive with JavaScript

Today's agenda — 1 h 45 min

30'

The DOM

Document tree, selecting elements, reading & changing content and styles

30'

Events

addEventListener, the event object, bubbling & delegation

35'

Hands-on build

Interactive To-Do list combining ES6+, DOM and events

10'

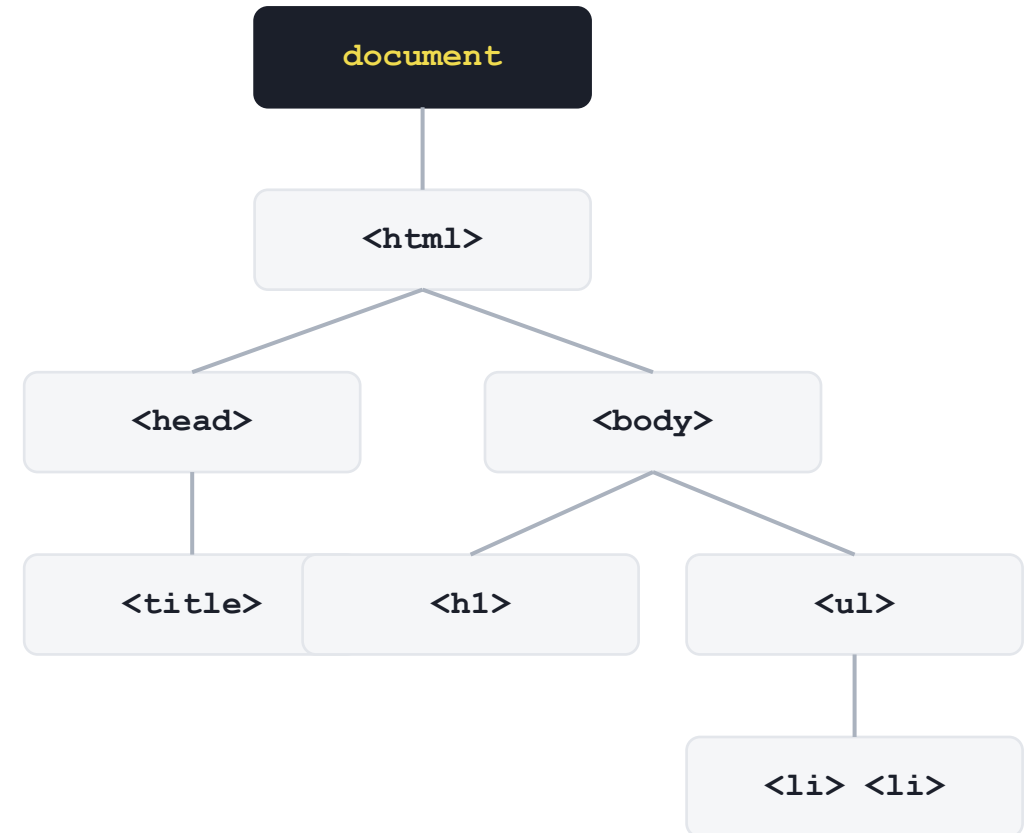
Wrap-up

Common pitfalls, homework, preview of Week 2 (async JavaScript)

The Document Object Model

- The browser parses your HTML into a live tree of objects — the DOM
- Every tag becomes an element node; text becomes text nodes
- JavaScript reads and changes the page through the document object
- Change the DOM → the browser re-renders instantly — no reload

Mental model: HTML is the blueprint; the DOM is the living building your JS can renovate at runtime.



Selecting elements

```
index.html

<ul id="list">
  <li class="item">Milk</li>
  <li class="item done">Bread</li>
</ul>
```

```
select.js

// modern selectors take CSS syntax
const list = document.querySelector("#list");
const first = document.querySelector(".item");
const all = document.querySelectorAll(".item");
// NodeList: all.forEach(el => ...)

// older but still common
document.getElementById("list");
```

querySelector(css)

first match — your default choice

querySelectorAll(css)

all matches as a NodeList

getElementById(id)

fast lookup by id (no # prefix)

el.querySelector(...)

search inside one element

Any CSS selector works: "ul > li", ".done", "[data-id='3']" ...

Reading & changing elements

```
modify.js

const h1 = document.querySelector("h1");

// content
h1.textContent = "Hello CSC220";
h1.innerHTML = "Hello <em>CSC220</em>";

// attributes & styles
const img = document.querySelector("img");
img.setAttribute("alt", "logo");
img.src = "logo.png";
h1.style.color = "crimson";

// classes - prefer this for styling
h1.classList.add("highlight");
h1.classList.remove("hidden");
h1.classList.toggle("dark");
```

- `textContent` for plain text — safe with user input
- `innerHTML` parses HTML — never feed it raw user input (XSS risk)
- Inline style works, but `classList` + CSS classes keeps styling in CSS
- `classList.toggle` is perfect for dark mode, menus, 'done' states

Security preview: XSS returns in Week 9 (Authentication & Security).

Creating, inserting & removing elements

```
create.js

const list = document.querySelector("#list");

// create
const li = document.createElement("li");
li.textContent = "Eggs";
li.classList.add("item");

// insert
list.append(li);           // end of list
list.prepend(li);         // start of list

// remove
li.remove();

// build from data (Session 1 skills!)
const items = ["Milk", "Bread", "Eggs"];
list.innerHTML = items
  .map(i => `- ${i}</li>`)
  .join("");

```

1

Create

`document.createElement("li")`

2

Configure

`textContent, className, attributes`

3

Insert

`append / prepend / before / after`

4

Remove

`el.remove()` when no longer needed

`map().join("")` renders arrays to HTML — the same idea React automates in Week 10.

PART 2

Events

Reacting to the user: clicks, typing, form submissions — listeners, the event object, bubbling and delegation.

click!

Listening with addEventListener

```
events.js

const btn = document.querySelector("#save");

btn.addEventListener("click", () => {
  console.log("Saved!");
});

const input = document.querySelector("#name");
input.addEventListener("input", e => {
  console.log("typing:", e.target.value);
});

const form = document.querySelector("form");
form.addEventListener("submit", e => {
  e.preventDefault(); // stop page reload
  // handle the data with JS instead
});
```

click

buttons, links, anything tappable

input / change

text fields as the user types / after edit

submit

forms — pair with `e.preventDefault()`

keydown / keyup

keyboard shortcuts, Enter to add

mouseover / mouseout

hover effects driven by JS

The event object

```
eventObject.js

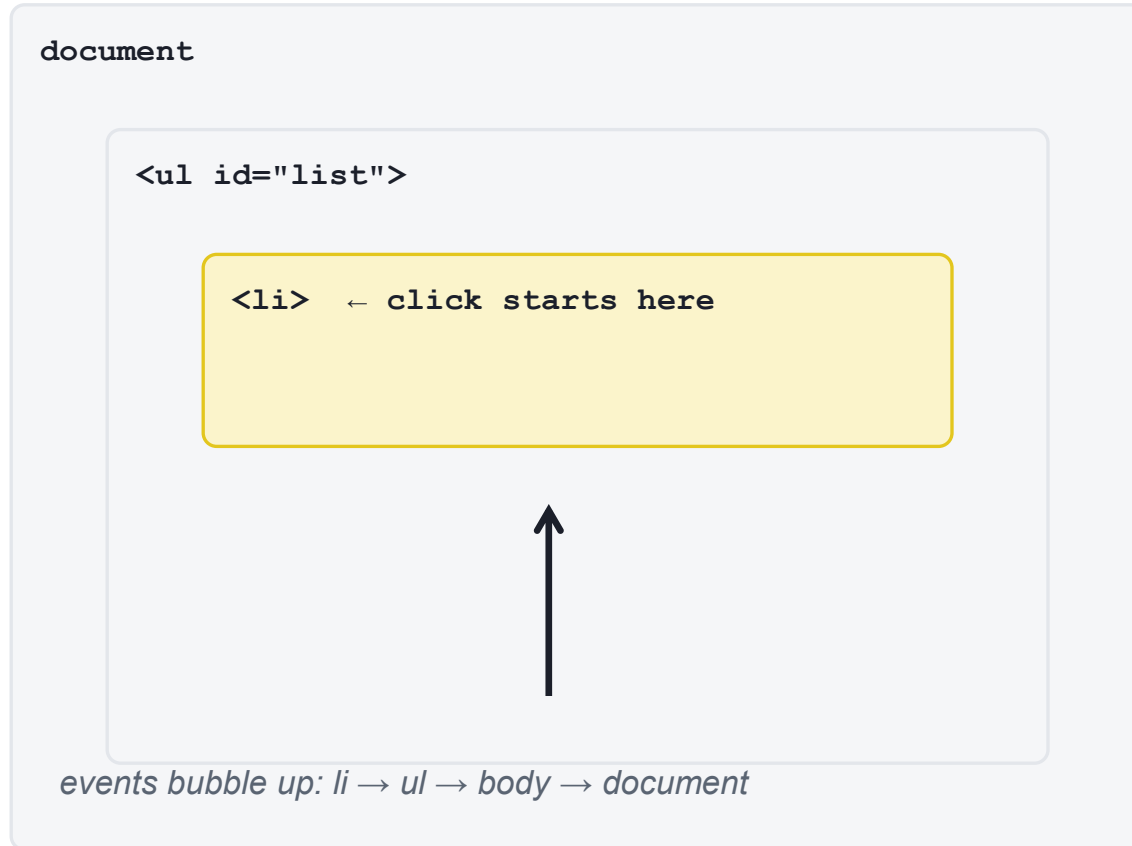
document.addEventListener("click", e => {
  console.log(e.type);      // "click"
  console.log(e.target);    // element clicked
});

form.addEventListener("submit", e => {
  e.preventDefault();      // stop default
  const { value } =
    document.querySelector("#task");
});

input.addEventListener("keydown", e => {
  if (e.key === "Enter") addTask();
});
```

- Every handler receives an event object (usually named `e`) automatically
- `e.target` — the actual element the event happened on
- `e.preventDefault()` — cancel the browser's default action (form reload, link navigation)
- `e.key`, `e.clientX/Y` ... — extra details depending on event type

Bubbling & event delegation



```
delegation.js

// one listener on the parent
// handles ALL <li> - even future ones
list.addEventListener("click", e => {
  const li = e.target.closest("li");
  if (!li) return;
  li.classList.toggle("done");
});
```

- Events fire on the target, then bubble up through every ancestor
- Delegation: listen once on the parent instead of on every child
- Works automatically for elements added later — perfect for dynamic lists

Build: interactive To-Do list (35 min)

```
index.html - starter

<form id="todo-form">
  <input id="task" placeholder="New task" />
  <button>Add</button>
</form>
<ul id="list"></ul>
```

```
app.js - step 2 shown

form.addEventListener("submit", e => {
  e.preventDefault();
  const text = input.value.trim();
  if (!text) return;
  const li = document.createElement("li");
  li.textContent = text;
  list.append(li);
  input.value = "";
});
```

Requirements

- 1. Select the form, input and list elements
- 2. On submit: prevent reload, create an `<li>`, append it
- 3. Click a task → toggle a `.done` class (use delegation)
- 4. Add a `×` button per task that removes it
- Stretch: task counter · 'Clear completed' · Enter-key focus

Use today's tools: arrow functions, template literals, `classList`, delegation. Submit via LMS before you leave.

Common pitfalls to avoid



Script runs before the DOM exists

querySelector returns null → use `<script defer>` or place the script at the end of `<body>`



innerHTML with user input

opens the door to XSS — use `textContent` for anything typed by users



Listening on each child element

breaks for dynamically added items — delegate from the parent instead



Forgetting `e.preventDefault()` on submit

the page reloads and your app state vanishes



`==` instead of `===`

coercion bugs that are painful to trace — keep the Session 1 rule

You can now make pages interactive

- 1 Explain the DOM and traverse it with `querySelector` / `querySelectorAll`
- 2 Read & modify content, attributes, styles and classes safely
- 3 Create, insert and remove elements from data arrays
- 4 Handle clicks, input and form submission with `addEventListener`
- 5 Use bubbling & delegation for dynamic elements

Homework

Finish the To-Do stretch goals. Reading: MDN 'Introduction to events' + Eloquent JavaScript ch. 14–15.

Next week — Asynchronous JavaScript

Callbacks, promises, `async/await` and the Fetch API — pulling live data from public APIs into the DOM skills you built today.